Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

ITEC 204 – Data Structures and Algorithms

Name: NACION, JASPER JESS C.	**Date: 10/4/2026**
Section:BS INFO TECH – 2A	**Instructor: MS. MAXYNE IGNACIO**

I. **Objective:** Write a short explanation of what your program is designed to do. (Minimum of 3 sentences)

Answer:

The primary objective of this program is to design and implement an IT Automation Incident Ticket Manager that stores, manages, and tracks active IT support incidents using Python. It utilizes a dynamic linear data structure to handle real-time modifications such as registering new incident tickets, searching for existing issues, and removing resolved cases. By executing these core operations, the program demonstrates how linear collections can effectively organize unstructured system alerts into an accessible, searchable, and structured format.

II. **Program Operations:** Briefly explain how your program performs each operation. (Insert Specific Code Line and Your Explanation)

Add Incident:

Code Line:
tickets.append({"id": incident_id, "bot": bot.strip(), "desc": description.strip()})
Explanation:

- This operation takes the user inputs for the Incident ID, Bot name, and Short Description, bundles them into a dictionary, and inserts it at the end of the tickets list using Python's built-in .append() method. Before doing so, it passes the ID to a validation loop to ensure all fields are filled and that no duplicate IDs exist.

Display Incidents:
Code Line: for i in range(len(tickets)): t = tickets[i]; print(f"{i + 1:<5}{t['id']:<14}{t['bot']:<20}{t['desc']}")
Explanation: This operation checks if the tickets list is empty. If it is not, a for loop iterates sequentially through the list index by index, formatting and printing each element's internal dictionary keys (id, bot, desc) into a clean, human-readable console table.

Search Incident:

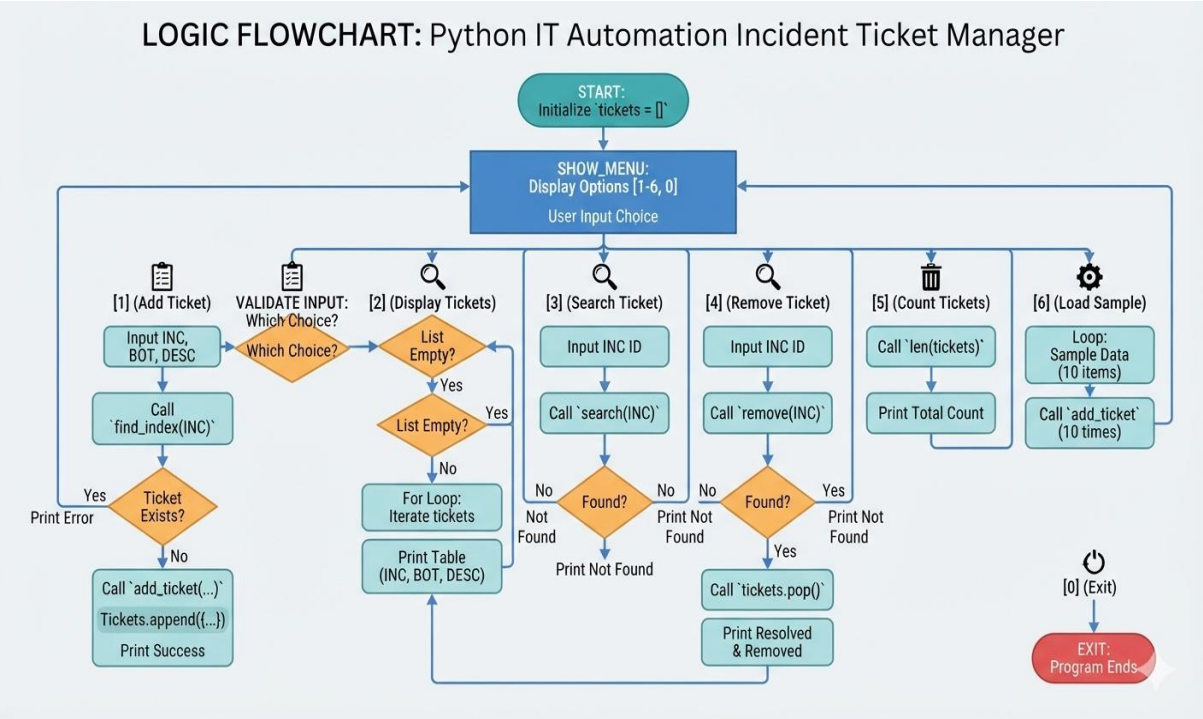
Code Line: for i in range(len(tickets)): if tickets[i]["id"] == incident_id.upper(): return i (Inside find_index)
Explanation: Finding a ticket relies on a classic **Linear Search** mechanism. The program loops sequentially from the first element to the last, comparing each dictionary's "id" key to the requested ID. If found, it returns its index to display the ticket details; otherwise, it returns -1 to signal that the ticket does not exist.

Remove Resolved Incident:
Code Line: removed = tickets.pop(index)
Explanation: Once an incident is marked as resolved, the program searches for its index position. If the ticket exists, it executes the .pop(index) method, which targets and extracts that specific element from the array list, shifting subsequent elements forward to maintain a gapless sequence.

-

Count Active Incidents:
Code Line: print(f"Total active incident tickets: {len(tickets)}")
Explanation: This function calculates the size of the current collection by invoking Python's native len() function on the tickets list array, displaying the absolute number of ongoing tickets directly to the user.

III. **Flowchart:** Create a flowchart showing the overall process and logic of your program.



IV. **Conclusion:** Briefly explain what you learned from the activity and how the linear data structure was applied.

Through this lab activity, I learned how to manipulate a dynamic array (Python List) to handle real-time transactional system data efficiently. A linear data structure was explicitly applied here through sequential positioning, where data elements are stored next to each other sequentially, allowing elementary operations like Linear Search (traversing items step-by-step) and Index Manipulation via appending and popping elements. This assignment reinforced that while linear structures are highly intuitive and easy to implement for managing dynamic queues, their search operations scale at a time complexity of \(O(n)\), highlighting the practical trade-offs involved in algorithm design.

V. .